

L0 PID Control Design Note

Purpose

L0 converts a planned waypoint into left and right differential-drive wheel speeds. The controller is intentionally small and deterministic so it can run at the simulator tick rate without introducing scheduling jitter.

Implementation: ``terrascout/control/pid.py``

Benchmark: ``benchmarks/control_benchmark.py``

State And Inputs

Robot pose:

$x = [p_x, p_y, \theta]$

Waypoint:

$w = [w_x, w_y]$

Wheel command output:

$u = [v_{\text{left}}, v_{\text{right}}]$

The rover model uses wheel base ``b = 0.72 m`` and wheel speed saturation at ``+/-1.6 m/s``.

Motion Model Used By The Simulator

The rover is advanced with the midpoint differential-drive update:

$v = (v_{\text{left}} + v_{\text{right}}) / 2$

```

omega = (v_right - v_left) / b
theta_mid = theta + 0.5 * omega * dt
x' = x + v * cos(theta_mid) * dt
y' = y + v * sin(theta_mid) * dt
theta' = wrap(theta + omega * dt)

```

Slip scales both wheel velocities by ``1 - slip_fraction`` before integration.

Controller

The waypoint bearing and heading error are:

```

theta_target = atan2(w_y - y, w_x - x)
e_theta = wrap(theta_target - theta)
d = hypot(w_x - x, w_y - y)

```

The linear target speed ramps down inside ``slow_radius_m``:

```

v_target = min(v_cruise, v_cruise * d / slow_radius)

```

If the heading error exceeds 0.3 rad, translation stops and the rover rotates toward the target. Speed feedback uses the measured encoder speed in mission runs, with the target speed as feedforward.

Each PID loop applies:

```

I_t = clamp(I_{t-1} + e_t * dt, -I_max, I_max)
D_t = (e_t - e_{t-1}) / dt
y_t = K_p * e_t + K_i * I_t + K_d * D_t

```

The controller maps linear and angular commands to wheel velocities:

```

v_left = v_linear - 0.5 * omega * b

```

```
v_right = v_linear + 0.5 * omega * b
```

Both outputs are clipped to the rover speed limits.

Pseudocode

```
function wheel_commands(pose, waypoint, dt, measured_speed):
    distance = norm(waypoint - pose.xy)
    heading_error = wrap(atan2(waypoint.y - pose.y, waypoint.x - pose.x) - pose.theta)
    speed_target = min(cruise_speed, cruise_speed * distance / slow_radius)
    if abs(heading_error) > 0.3:
        speed_target = 0
        speed_pid.reset()
    linear = max(0, speed_target + speed_pid.update(speed_target - measured_speed, dt))
    if speed_target == 0:
        linear = 0
    angular = heading_pid.update(heading_error, dt)
    return clip(linear - angular * b / 2), clip(linear + angular * b / 2)
```

Acceptance Evidence

The L0 benchmark runs randomized slip trials and reports maximum cross-track error, heading settle time, and heading overshoot. Current acceptance tests require straight-line cross-track error below the project threshold and bounded heading response across seeded runs.

References

- Astrom and Murray, Feedback Systems: An Introduction for Scientists and Engineers.
- Siciliano et al., Robotics: Modelling, Planning and Control, differential-drive kinematics.